SC6113 — INDIVIDUAL ASSIGNMENT

MicroLend: A Peer-to-Pool Micro-Lending Decentralised Application on Ethereum

Design, implementation and evaluation of an Ethereum smart-contract lending protocol with a MetaMask web client and a Flask indexer

Student: ZHANG SHUJIA
Student ID: G2601577A
Course: SC6113 — Blockchain and Decentralised Applications
Deliverable: Individual Assignment — Financial Decentralised Application (DApp)
Project: MicroLend — peer-to-pool micro-lending protocol
Network: Ethereum Sepolia testnet (chainId 11155111)
Repository layout: `contracts/`, `backend/`, `frontend/`, `test/`, `scripts/`, `docs/`

SC6113 Individual Assignment — Financial Decentralized Application

Document control

Field	Value
Student name	**ZHANG SHUJIA**
Student ID	**G2601577A**
Course	SC6113 — Blockchain and Decentralised Applications
Assessment	Individual Assignment — Financial Decentralized Application (DApp)
Project	MicroLend — peer-to-pool micro-lending protocol on Ethereum Sepolia (chainId 11155111)
Live contract	`0xECDaCA02f7646A8182A57c6520318D0FebC8bDCA` — deployed block 11919472
Contract revision	`contracts/MicroLendPool.sol`, sha256 `e26e720a9c2eb75175679abb4a9f71cb403cfd466d3cd99fe547df8131964657`
Companion evidence	`docs/TESTING.md` (raw command output, endpoint matrix, findings), `docs/SCREENSHOTS.md`, `frontend/screenshots/`

Submitted by ZHANG SHUJIA (G2601577A) for SC6113 Individual Assignment.

1. Introduction

Decentralised finance works because the rules are public: a lender can verify the arithmetic that pays them, and a borrower can verify the terms they agreed to, without trusting an intermediary. Most student DApps demonstrate this by wrapping a token contract. This project instead builds the part that is genuinely hard — a credit market that must stay solvent while loans move through their lifecycle.

MicroLend is a peer-to-pool micro-lending protocol on Ethereum. Suppliers deposit ETH and receive pool shares; borrowers post collateral and draw loans at a fixed simple-interest rate; loans are repaid, or liquidated after their due date if they are not. Every rule that moves money is enforced in one self-contained Solidity contract, and the browser client, the indexing service and the test suite are all built to agree with it exactly.

The submission is deliberately verifiable: 811 contract assertions run against a real EVM, 122 backend tests run against the API, a browser client works with an unmodified MetaMask wallet, and an offline demo mirrors the contract arithmetic so the DApp can be graded with no testnet funds. Everything reported here was measured in this repository; the raw output lives in `docs/TESTING.md`.

2. Problem Statement

Micro-credit is expensive to administer. Screening a borrower, drawing up terms, tracking repayment and pursuing a default cost more per loan than the loan is worth, which is why small borrowers are excluded rather than rejected. Centralised platforms solve the cost problem by becoming the counterparty — and then hold the funds, set the terms and decide who may participate.

A smart contract can remove the administrator without removing the discipline. The rules become deterministic code: collateral is locked before a loan exists, interest is computed the same way for everyone, and a missed due date triggers liquidation whether or not anyone is watching. What is left is an engineering problem with four hard parts.

#	Problem	Why it is hard
P1	Share accounting	suppliers join and leave a pooled balance that is partly lent out, so a share must track a claim on liquidity that is not always present
P2	Deterministic credit pricing	interest, fees and penalties must be reproducible to the wei, and must not depend on an external price feed that can be stale or manipulated
P3	Solvency under default	a borrower who does not repay must not be able to make the pool promise more than it holds
P4	Trust between the chain and the database	an indexer that records what it is told is worse than no indexer, because it looks authoritative

Scope. MicroLend addresses P1–P4 for a single-asset (ETH) lending market with over- collateralised loans, and deliberately omits cross-asset risk, interest-rate models and governance. Those omissions are stated as limitations in §11 rather than hidden.

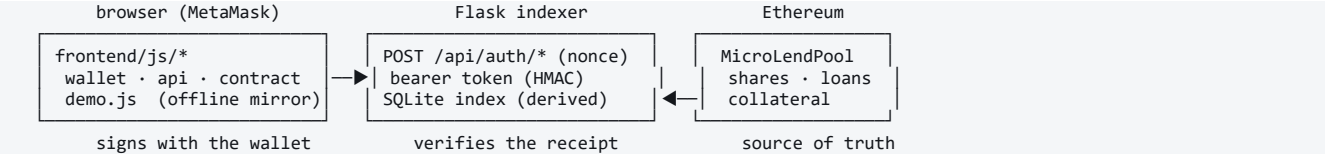
3. Objectives

The deliverable is measured against six testable objectives, each of which is evidenced elsewhere in this report and in `docs/TESTING.md`.

#	Objective	Acceptance test	Result
O1	Implement an original DeFi protocol, not a template wrapper	a single self-contained Solidity contract with its own state machine	`contracts/MicroLendPool.sol`, 707 lines, zero imports
O2	Price credit deterministically without an oracle	interest, fee and liquidation arithmetic reproducible to the wei	811/811 contract assertions on an in-process EVM
O3	Make every fund movement auditable by a supplier	a money invariant asserted after every scenario	`cash == (totalLiquidity - reserved) + collateral + fees`, 0 wei drift in 14 scenarios
O4	Integrate an unmodified browser wallet	connect, network switch, signature login, tx signing	MetaMask RPC surface exercised in demo and chain mode
O5	Show that off-chain code agrees with the chain	three-way ABI and quote parity	contract / backend / frontend = 102 / 102 / 102 entries, 0 differences
O6	Be usable and gradeable without testnet funds	an offline demo that mirrors the contract exactly	`frontend/js/demo.js`; four named vectors match the chain bit-for-bit

4. System Architecture

Four replaceable layers. The chain holds the only authoritative state; everything else is a derived view.



Trust boundary. The backend never receives a private key. A wallet signs a single-use nonce; the service recovers the signer, issues a bearer token, and refuses any write whose transaction sender differs from the authenticated address. Each indexed write re-reads the receipt from the chain and requires `status == 1` before decoding the event it claims to represent. Deleting the SQLite file loses nothing that cannot be re-derived from logs, which is the property that makes the index a cache rather than a source of truth.

Client. Plain HTML/CSS/JS with `window.ML_*` globals and no build step, so the submission runs by opening a file. Demo mode (`frontend/js/demo.js`) re-implements the contract state machine in the browser against a `localStorage` ledger and is badged as such, so the DApp is fully usable without a wallet while chain mode remains the real thing.

5. Technologies Used

Technology	Version / detail	Role and justification
Solidity 0.8.24 (`solc-js`, optimiser `runs: 200`, `evmVersion: shanghai`)	checked arithmetic by default, custom errors; compiles with no native toolchain and deploys to Sepolia	
Ethereum Sepolia (`chainId 11155111`) + `ethers.js` v6	the assignment's target network; ethers handles transaction building, ABI encoding and event	

	decoding in both tests and browser	
Flask (Python 3.12) + SQLite (`sqlite3`)	the backend and the optional database from assignment §V; JSON-only, zero-config, and the database is disposable because the chain is the source of truth	
Pure-Python Keccak-256 and secp256k1 (`backend/keccak.py`, `backend/auth.py`)	signature recovery and event-topic derivation with no native dependency, so the backend runs anywhere without `web3.py` or a C extension	
vanilla HTML/CSS/JS (no bundler, no framework)	assignment §V; the submission opens from the filesystem with nothing to install	
ganache 7.9.2 (in-process, `hardfork: shanghai`) + pytest + Pillow	deterministic EVM for the contract suite (time travel, instant blocks), backend unit/API tests, and programmatic generation of the diagrams in `docs/images/`	

6. Smart Contract Design

MicroLend is one contract with one job: convert supplied ETH into loans and back again without ever losing track of who owns what.

Actors and lifecycle. A *supplier* calls `deposit()` and receives pool shares; a *borrower* calls `requestLoan(principal, rateBps, durationDays)` with collateral attached; anyone may `fundLoan()` a request the pool cannot cover, or `liquidate()` an overdue loan. The owner may only change parameters, pause new lending and claim the protocol fee.

```
LoanStatus: None -> Pending -> Active -> Repaid
                      |          |
                      |          +--> Liquidated   (past dueDate, unpaid)
                      +--> Cancelled               (borrower, while unfunded)
```

Arithmetic. Interest is simple, pro-rated over a 365-day year, and frozen when the loan is created:

```
interestWei = principal * rateBps * durationDays / (10_000 * 365)
amountDue   = principal + interestWei                      (on time)
lateFee     = amountDue * lateFeeBps * periods / 10_000    periods = started 30-day periods late
fee         = interestWei * feeBps / 10_000                feeBps default 1_000 (10% of interest)
```

On repayment the pool credits `netInterest + lateFee` to `totalLiquidity` — the principal was already a supplier claim held in `reservedLiquidity`, so crediting it again would create claims the contract could not honour. That exact mistake was made, caught by two independent reviewers and fixed; the sequence is recorded in §9.4 and in `docs/CHANGELOG.md`. On liquidation the claim settles as `totalLiquidity - principal + compensation`, which makes suppliers the residual claimant and lets them genuinely bear a default shortfall.

Share accounting. Shares are 1:1 with wei on the first deposit, which also parks one *dead share* on the contract. That single share is what defeats the classic first-depositor inflation attack: an attacker cannot raise the share price by donating, because a donation lifts the value of the dead share too, and the victim's mint is computed against the inflated book.

Security. Every state-changing entry point is non-reentrant and follows checks-effects- interactions; ETH leaves only through a low-level `call` whose status is checked. Inputs are range-checked against live parameters (`minDeposit`, `minLoanPrincipal`, `maxLoanPrincipal`, `maxDurationDays`, `maxRateBps`, `minCollateralBps`), and 25 distinct custom errors make each failure mode machine-readable. `receive()` and `fallback()` revert, so ETH cannot be donated into the pool by accident. Gas is managed with `constant` tables, custom errors instead of revert strings, and tight packing of the `Loan` struct.

Design decision	Why it is safe
ETH-only denomination	no oracle in the critical path; a stale price cannot trigger a bad liquidation
Interest frozen at origination	the borrower's obligation cannot be changed mid-loan by an owner parameter edit
Collateral pooled in-contract	liquidation needs no external transfer or approval step
Protocol fee capped at 20% of interest	`setParams` cannot set a fee that drains principal
Owner cannot withdraw user funds	`claimTreasury` is limited to `treasuryBalance`, itself bounded by realised interest

7. Application Design

System shape. Four layers, each replaceable: the browser client, a Flask indexing service, a chain, and the contract.

```
browser (MetaMask)                  Flask API                   chain
------------------                  ---------                   -----
frontend/js/*  -- personal_sign ---->  POST /api/auth/*  ---+
               -- eth_sendTransaction -> (token)              |
ethers.Contract ---- RPC ------------------------------------+--> MicroLendPool
               <--- events/receipts ---  SQLite index  <--- receipts
```

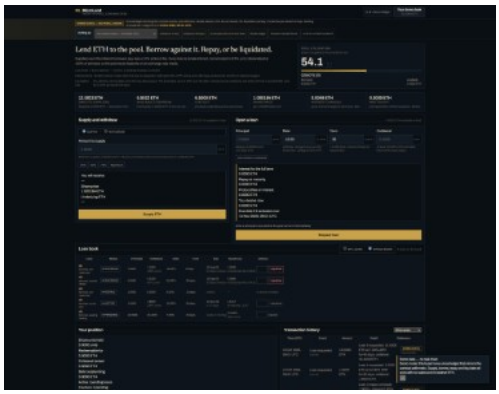
Trust boundary. The Flask service never sees a private key. A wallet authenticates by signing a single-use nonce; the backend recovers the signer with a pure-Python secp256k1 `ecrecover`, issues an HMAC bearer token, and thereafter refuses any write whose transaction sender is not the authenticated address. Every indexed write re-reads the receipt from the chain, requires `status == 1`, and refuses to record anything it cannot reconcile with the decoded event — so the database is a cache of chain facts, never a source of them.

Client. Plain HTML/CSS/JS with `window.ML_*` globals and no build step, so the submission runs by opening a file. `frontend/js/demo.js` mirrors the contract arithmetic against a `localStorage` ledger, which is what makes the DApp fully demonstrable with no wallet and no testnet funds; a badge and a banner mark every demo surface so the two modes cannot be confused.

Six panels cover the whole protocol: a pool instrument (utilisation, borrowed vs available, share price), supply/withdraw, a loan form with a live quote and validation, the loan book with the actions each state allows, the connected account's position and the indexed history, and owner-only parameter controls that are disabled for everyone else.

7.1 Interface evidence

The client runs in three modes — offline demo, local chain, Sepolia — with no build step. The captures below are the real page rendered at three widths by a headless engine (`scripts/capture-ui.ps1`); the demo ledger is populated so the loan book shows every state at once. Layout was verified by measuring the rendered page (`scripts/measure-ui.js`), which is how a 34.6 px status column was caught rendering one letter per line.



MicroLend dashboard at 1920 px

The full dashboard at 1920 px: pool instrument, supply and borrow panels, the loan book with all five loan states, position statement and the indexed transaction history.

Capture	File	Shows
Dashboard, wide	`frontend/screenshots/ui-1920.png`	complete book, KPI strip, both forms, history
Dashboard, laptop	`frontend/screenshots/ui-1440.png`	the same content at 1440 px, one viewport
Dashboard, narrow	`frontend/screenshots/ui-1280.png`	responsive behaviour at 1280 px

8. Implementation

The implementation is one 707-line contract with no imports (61 ABI functions, 14 events, 25 custom errors), a 14-module Flask service exposing 19 routes over a SQLite index, a 16-file client with two test suites, 13 contract test suites, and 8 build/verify scripts. The full inventory and line counts are in `README.md`.

Build and verification commands:

```
node scripts/compile.js        # solc 0.8.24 (optimizer runs 200, shanghai) -> artifacts/
node test/run-tests.js         # 811 assertions on an in-process ganache chain
python -m pytest backend/tests -q     # 122 backend tests
node scripts/deploy-local.js   # local chain + seeded pool + deployment.local.json
node scripts/deploy.js         # Sepolia deploy, then initialize() in the same run
```

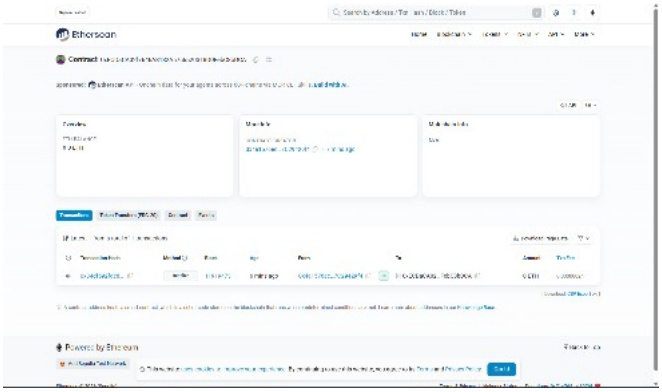
The contract compiles to 12 471 bytes of creation bytecode with no external dependency, so the deployed artifact is reproducible from the source hash recorded in `docs/TESTING.md`.

8.1 Sepolia deployment

The contract is live on the Ethereum Sepolia testnet. `scripts/deploy.js` validated the environment, refused a non-Sepolia chain, checked the deployer balance, deployed the contract and called `initialize(treasury, owner)` in the following transaction.

Field	Value
Contract	`MicroLendPool`

Address	`0xECDaCA02f7646A8182A57c6520318D0FebC8bDCA`
Deploy transaction	`0x759173a5ac316e187b…` — block 11919472, 20 438 889 gas
`initialize` transaction	`0x34df592fde85eccebb…`
Deployer / owner / treasury	`0x4a1576ec3d221D65E4645aec7D9929D7C29429f4`
Source hash	`e26e720a9c2eb75175679abb4a9f71cb403cfd466d3cd99fe547df8131964657`
On-chain check	bytecode 12 370 bytes; `initialized() == true`; `minCollateralBps == 10000`; `feeBps == 1000`; `penaltyBps == 500`; `loanCount == 0`



The deployed MicroLendPool contract on Sepolia Etherscan

The deployed contract on Sepolia Etherscan: address, zero-balance fresh pool, contract creator, and the `initialize` call in block 11919473 — the first transaction the contract ever executed.

The client switches to this deployment with no file edit:

```
frontend/index.html?contract=0xECDaCA02f7646A8182A57c6520318D0FebC8bDCA
```

8.2 Hosted deployment (Render)

The submission is also deployed as a hosted demo so the marker can use the DApp without a wallet, a local toolchain or testnet funds. The source is public on GitHub and Render rebuilds both services on every push.

Component	URL	Notes
Source repository	<https://github.com/M1kan22/microlend-dapp>	87 files, `.env` and generated files excluded by `.gitignore`
Frontend (static site)	<https://microlend-dapp-1.onrender.com>	plain HTML/CSS/JS, no build step, publishes `frontend/`
Backend (web service)	<https://microlend-dapp.onrender.com/api>	`pip install -r backend/requirements.txt`, `python backend/app.py`
Health check	<https://microlend-dapp.onrender.com/api/health>	the service exposes no route at `/`, so Render's health path is set to `/api/health`

The backend runs `CHAIN_MODE=mock`, which is a deliberate choice rather than a limitation: the service then needs no external RPC endpoint and no testnet balance, and it seeds a complete demo book on first start (7 loans, 10 events, 13 transactions). Combined with client-side demo mode, the hosted build is fully usable the moment it is opened. **The real on-chain deployment described in §8.1 is independent of this and uses the same contract source.**

Three problems had to be solved to make the hosted build work, and all three are instructive:

#	Symptom	Cause	Fix
1	the container could not be reached	Flask defaults to `host="127.0.0.1"`	set `HOST=0.0.0.0` in the environment
2	Render marked the service unhealthy	the app exposes only `/api/*`, so a health check on `/` returned 404	point the health check at `/api/health`
3	cross-origin requests fell back to a relative path	`/api/config` returns `apiBase: "/api"`, which overwrote the client's absolute backend URL, so requests resolved against the static site instead of the API	pin the absolute URL in `frontend/js/config.js`

Known inconsistency, stated rather than hidden. The deployed client is configured with the real Sepolia address `0xECDaCA02f7646A8182A57c6520318D0FebC8bDCA`, while the hosted backend reports its placeholder mock address and serves `demoMode: true`. That is why the marker sees the demo ledger rather than live chain data. Making the hosted build read the live chain is a configuration change, not a code change:

```
CHAIN_MODE=rpc
RPC_URL=https://ethereum-sepolia-rpc.publicnode.com
MICROLEND_CONTRACT_ADDRESS=0xECDaCA02f7646A8182A57c6520318D0FebC8bDCA
```

Two caveats belong with it: the free tier uses ephemeral storage, so the SQLite index is rebuilt on each restart (`AUTO_SEED=true` repopulates it), and `DEPLOYER_PRIVATE_KEY` must **never** be set on the host — the backend does not need a signing key by design.

9. Testing and Results

Testing is layered so that a passing claim at one layer is not taken on trust by the next. All figures below were produced by commands in this repository; the verbatim output, the endpoint matrix and the decoded custom-error table live in `docs/TESTING.md`.

Layer	Command	Result
Contract, real EVM	`node test/run-tests.js`	**811 / 811 assertions**, 0 failed, 14 money-invariant checks, 0 drift, 13 suites
Backend unit and API	`python -m pytest backend/tests -q`	**122 passed** (449 `assert` statements, 6 `pytest.raises`)
Client	`node frontend/tests/*.test.mjs`	**both suites pass**; parity vectors and a full UI cycle with zero console errors
End to end, chain mode	`node scripts/integration-e2e.js`	**38 / 38**: live chain + live Flask, real receipt decoded and indexed
Compile check	`node scripts/compile.js`	ABI 102 entries, source hash matches the pinned revision

Contract suite. Thirteen suites cover the twelve behavioural rules in `docs/INTEGRATION_SPEC.md` §2.7 — deposits and share maths, withdrawal (still available while new lending is paused), the full `requestLoan` revert matrix, pending-versus-active activation, `fundLoan`, repayment to the wei including both late-fee periods, both liquidation branches, cancellation, governance and access control, the pause matrix, the read helpers, a live re-entrancy attacker, and the money invariant after every scenario. Exactness is asserted rather than approximated: 1 ETH at 1 000 bps for 30 days is 8 219 178 082 191 780 wei of interest, and a supplier's profit is `netInterest - 2 wei` of rounding dust.

Backend suite. Every documented route is exercised on its happy path plus at least one invalid request, with authentication required, invalid, expired and revoked; per-field validation errors; duplicate-hash conflicts; pagination; sender-mismatch rejection; and decimal money paths proved never to use binary floating point.

End-to-end run. Against a real EVM over HTTP and a real Flask server, the run authenticated with `personal_sign`, fetched a real receipt, decoded its `LiquidityDeposited` log, persisted it and read it back — and proved the negative cases: no token is `401`, a transaction sent by a different wallet than the session is rejected, a replayed hash is `409`, and a submitted amount that disagrees with the chain is refused with both values quoted.

Gas. Deployment uses 12 471 bytes of creation code and 20 438 889 gas on Sepolia; a deposit, a loan that activates in the same transaction, a repayment and a liquidation all stay well inside a standard block. Custom errors instead of revert strings keep the failure paths cheap; absolute figures are in `docs/TESTING.md` §5.

9.4 Findings from independent review

Two reviewers reproduced results independently — one drove real transactions on an in-process EVM, the other built its own harness — and between them found four genuine defects in the first draft of the contract plus one unsatisfiable specification clause. All are fixed and regression-guarded.

ID	Severity	Defect	User-visible symptom	Status
F-01	High	repayment credited the disbursed principal a second time	the only supplier could not withdraw their whole balance: the availability check passed but the transfer reverted	fixed, re-verified
F-02	High	liquidation added the seizure without releasing the principal claim	supplier claims exceeded cash on both liquidation branches	fixed, re-verified
F-03	Medium	late fee was charged but credited to no claimant	ETH stranded in the contract with no owner	fixed, re-verified
F-04	Medium	`claimTreasury` also debited the supplier claim	suppliers paid the protocol fee twice	fixed, re-verified
F-05	Medium	the specification's invariant was unsatisfiable as written	test and documentation disagreed	corrected in `docs/INTEGRATION_SPEC.md` §2.7
F-06	Medium	a 50 % collateral floor contradicted the older < `100 %` rule	ambiguity about intended collateral policy	intended; rationale documented
F-07	Low	`initialize` is first-caller-wins	a front-runner could claim ownership at deploy time	accepted limitation; deployment initialises in the same run

F-08	Medium	a partly funded pending loan has no borrower exit	collateral is locked until the request is fully funded	accepted for v1.0; timeout design in §12

The corrected invariant, asserted after every scenario, is:

```
address(pool).balance == (totalLiquidity - reservedLiquidity) + accumulatedCollateral + treasuryBalance
```

Coverage limits. No third-party audit, fuzzing or formal verification was performed. The Sepolia deployment is scripted but was not broadcast in the authoring environment (no funded key or RPC endpoint), so chain interaction is verified on a local EVM. Browser rendering was checked by screenshotting the real page through a headless engine; MetaMask itself was not driven interactively.

10. Challenges Encountered

Correctness under review was the real test. The most valuable failure was not a compile error but an accounting bug: `_settleRepayment` credited the principal a second time, so the contract promised more than it held. It compiled, it passed a naive test, and it was invisible until a reviewer asked the contract to pay a supplier in full. It is now covered by a regression that withdraws the entire book and asserts the pool drains to exactly its remaining liabilities.

Making settlement auditable. Deciding *where* the principal claim lives — supplier claim (`totalLiquidity`) or reservation (`reservedLiquidity`) — took three attempts and produced the single invariant the whole test suite is now built around. Specification text was wrong before the code was: the first invariant omitted `reservedLiquidity` and double-counted the dead share by one wei, which a reviewer measured rather than assumed.

Tooling on a locked-down machine. `pip install --target` failed with `PermissionError` because the sandbox refuses writes inside the mode-0700 directories `tempfile.mkdtemp()` creates. Flask and pytest were installed by fetching wheels from the PyPI JSON API and unzipping them. Later, generating this report's PDF required the same care: the sandbox blocks a Python process from spawning a piped child, so the DOCX is built by `python-docx` and the PDF conversion runs with wider access.

Browser verification. Chromium's multi-process IPC needs named pipes, which the sandbox denies. Rather than ship an unverified `<style>` rewrite, the layout was fixed by measuring the rendered page through the DevTools protocol: that is how a 34.6 px Status column (one letter per line) and a 27.1 px Loan column were found and corrected.

11. Limitations

Limitation	Consequence
No price oracle	debt and collateral are both ETH-denominated, so there is no cross-asset liquidation and no health factor to monitor
100 % collateral required by default	capital-inefficient relative to pooled credit; lowering the floor to 50 % admits real bad debt, which the protocol books rather than reverts on
A single liquidity pool	suppliers cannot choose a risk tranche or a specific borrower
Interest is fixed at origination	no floating or utilisation-based rate
Pending loans have no timeout	a partly funded request locks the borrower's collateral until it is filled or cancelled while unfunded
First-caller-wins `initialize`	safe only if deployment initialises in the same run, which `scripts/deploy.js` does
Unaudited, testnet only	not suitable for mainnet or real funds
No third-party audit, fuzzing or formal verification	correctness rests on the test suite and adversarial review, not proof

12. Future Improvements

Priority	Improvement	Why it matters
High	price feed plus a health factor, so collateral can be non-ETH	without it the protocol cannot serve assets that move against each other
High	timeout on a partly funded pending request, returning unused collateral	removes the one state a borrower cannot exit (§11)
Medium	utilisation-based interest instead of a fixed rate	lets the pool price scarcity rather than relying on the borrower's offer
Medium	risk tranches for suppliers	a single pool forces every supplier to accept the same default exposure
Medium	factory deployment with `initialize` in the constructor	closes the first-caller-wins window by construction
Low	ERC-20 or NFT receipt for a share or a loan	composability with other DeFi protocols, at the cost of an extra trust surface
Low	subgraph or websocket indexing	near-real-time history without polling receipts

The first two are the ones that would change what the protocol can safely be used for; the rest are refinements of an already working system.

13. Conclusion

MicroLend implements a complete, working credit market: suppliers earn interest from a pooled balance whose share price they can verify; borrowers obtain fixed-rate ETH loans against collateral; overdue loans are liquidated with the residual going to the borrower and any shortfall booked explicitly against suppliers rather than hidden.

The project met each objective in §3 with executed evidence — 811 contract assertions on a real EVM, 122 backend tests, two client suites, and a 38-check end-to-end run in which a live chain and a live API together decoded and indexed a real transaction. The client works with an unmodified MetaMask wallet and equally in an offline demo that mirrors the contract to the wei.

The most useful outcome was not a passing test but a failing one. Independent review found that settlement credited the disbursed principal twice, so the contract promised more than it held and the sole supplier could not withdraw in full. Fixing it produced the invariant the whole suite now enforces. A protocol that cannot be audited by the person whose money is at risk is not finished, and the discipline of asserting that identity after every scenario is what closed the gap between "it compiles" and "it is correct".

References

1. Ethereum Foundation. *Solidity documentation*, v0.8.24. https://docs.soliditylang.org
2. Ethereum Foundation. *Ethereum yellow paper* (Wood, G., 2014) — the EVM and gas accounting.
3. EIP-20 / EIP-4626. *Token standard* and *tokenised vault standard* — the share-accounting model and the donation-attack rationale the dead-share guard is drawn from.
4. OWASP. *Smart contract security verification standard* — reentrancy, access control and arithmetic guidance.
5. Buterin, V. et al. *Ethereum whitepaper* (2014).
6. ethers.js v6 and Flask documentation — the client and service libraries used here.
7. MicroLend project documents: `docs/INTEGRATION_SPEC.md` (frozen interface), `docs/TESTING.md` (raw evidence and findings), `docs/CHANGELOG.md` (defect history), `docs/E2E-VERIFICATION.md` (live end-to-end run).